

Neo4j构建语义图谱，大模型秒懂表关系，Text2SQL准确率狂飙300%！告别瞎猜！

原创 大模型应用开发 大模型应用开发实战 2025年9月21日 16:00 浙江

用 Neo4j 图数据库优化 Text2SQL —— 从零构建语义关系图谱

微信公众号 | 搜一搜： 大模型应用开发实战

作者：AI 工程师实战派

发布时间：2025 年 9 月 21 日

在大模型驱动的智能数据时代，Text2SQL 的准确性严重依赖对 表结构 和 表间关系 的理解。传统方法依赖 Prompt 工程或 Schema 描述，效果不稳定。

本文带你用 Neo4j 构建“数据库语义图谱”，让大模型“看得懂关系”，显著提升 SQL 生成准确率！

💡 核心思想：将 MySQL 表结构 + 人工定义的关系 → 写入 Neo4j → 供大模型检索/推理 → 生成更精准 SQL

运行效果

00:26

本文流程一览

1.  环境配置 —— 安装依赖 & 配置数据库连接
2.  自动提取表结构 —— 从 MySQL 动态获取字段、主键、外键
3.  人工定义表关系 —— 补充业务语义，构建完整图谱
4.  写入 Neo4j —— 创建节点、关系、约束
5.  一键运行主函数 —— 自动化构建图谱
6.  开源项目推荐 —— 快速集成到你的大模型应用！

1 环境配置

1.1.安装neo4j

```
docker run -d \  
--name neo4j-apoc \  
-p 7474:7474 \  
-p 7687:7687 \  
-v ./volume/neo4j/data:/data \  
-v ./volume/neo4j/plugins:/plugins \  
-e apoc.export.file.enabled=true \  
-e apoc.import.file.enabled=true \  
-e apoc.import.file.use_neo4j_config=true \  
-e NEO4J_AUTH=neo4j/neo4j123 \  
neo4j:5.26.11-ubi9
```

1.2.安装必要依赖

```
pip install pymysql py2neo
```

1.3.MySQL 配置

```
MYSQL_CONFIG = {  
    "host": "localhost",  
    "port": 13006,  
    "user": "root",  
    "password": "your_password", # ← 替换为你的密码  
    "database": "text2sql_db",  
    "charset": "utf8mb4",  
}
```

1.4.Neo4j 配置

```
NEO4J_URI = "bolt://localhost:7687"
NEO4J_USER = "neo4j"
NEO4J_PASSWORD = "your_password" # ← 替换为你的密码
```

1.5.数据库连接函数

```
from py2neo import Graph
import pymysql

def connect_mysql():
    return pymysql.connect(**MYSQL_CONFIG)

def connect_neo4j():
    return Graph(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))
```

1.6.执行数据库脚本

```
DROP TABLE IF EXISTS `t_customers`;
CREATE TABLE `t_customers` (
  `customer_id` int NOT NULL AUTO_INCREMENT COMMENT '客户ID',
  `customer_name` varchar(100) NOT NULL COMMENT '客户姓名',
  `phone` varchar(20) DEFAULT NULL COMMENT '联系电话',
  `email` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL COMMENT '地址',
  `address` text COMMENT '地址',
  `city` varchar(50) DEFAULT NULL COMMENT '城市',
  `country` varchar(50) DEFAULT '中国' COMMENT '国家',
  `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `updated_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`customer_id`),
  UNIQUE KEY `email` (`email`)
) ENGINE=InnoDB AUTO_INCREMENT=11 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='客户表';

DROP TABLE IF EXISTS `t_order_details`;
CREATE TABLE `t_order_details` (
  `detail_id` int NOT NULL AUTO_INCREMENT COMMENT '明细ID',
  `order_id` int NOT NULL COMMENT '订单ID',
  `product_id` int NOT NULL COMMENT '产品ID',
  `quantity` int NOT NULL COMMENT '销售数量',
  `unit_price` decimal(10,2) NOT NULL COMMENT '销售时的单价',
  `line_total` decimal(12,2) NOT NULL COMMENT '行小计 (quantity * unit_price)',
```

```

        `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
        PRIMARY KEY (`detail_id`),
        UNIQUE KEY `uk_order_product` (`order_id`,`product_id`),
        KEY `product_id` (`product_id`)
    ) ENGINE=InnoDB AUTO_INCREMENT=38 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='订单明细'

DROP TABLE IF EXISTS `t_order_details`;
CREATE TABLE `t_order_details` (
    `detail_id` int NOT NULL AUTO_INCREMENT COMMENT '明细ID',
    `order_id` int NOT NULL COMMENT '订单ID',
    `product_id` int NOT NULL COMMENT '产品ID',
    `quantity` int NOT NULL COMMENT '销售数量',
    `unit_price` decimal(10,2) NOT NULL COMMENT '销售时的单价',
    `line_total` decimal(12,2) NOT NULL COMMENT '行小计 (quantity * unit_price) ',
    `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    PRIMARY KEY (`detail_id`),
    UNIQUE KEY `uk_order_product` (`order_id`,`product_id`),
    KEY `product_id` (`product_id`)
) ENGINE=InnoDB AUTO_INCREMENT=38 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='订单明细'

DROP TABLE IF EXISTS `t_order_details`;
CREATE TABLE `t_order_details` (
    `detail_id` int NOT NULL AUTO_INCREMENT COMMENT '明细ID',
    `order_id` int NOT NULL COMMENT '订单ID',
    `product_id` int NOT NULL COMMENT '产品ID',
    `quantity` int NOT NULL COMMENT '销售数量',
    `unit_price` decimal(10,2) NOT NULL COMMENT '销售时的单价',
    `line_total` decimal(12,2) NOT NULL COMMENT '行小计 (quantity * unit_price) ',
    `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    PRIMARY KEY (`detail_id`),
    UNIQUE KEY `uk_order_product` (`order_id`,`product_id`),
    KEY `product_id` (`product_id`)
) ENGINE=InnoDB AUTO_INCREMENT=38 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='订单明细'

```

2 自动提取 MySQL 表结构（智能识别主外键）

```

def get_tables_from_database(connection):
    """自动扫描数据库，提取表名、字段、主键/外键标记"""
    tables = {}
    with connection.cursor() as cursor:
        cursor.execute("SHOW TABLES")
        table_names = [row[0] for row in cursor.fetchall()]

        for table_name in table_names:

```

```

cursor.execute(f"SHOW COLUMNS FROM {table_name}")
columns = cursor.fetchall()
fields = []
for col in columns:
    field_name = col[0]
    key_type = col[3] # PRI=主键, MUL=外键候选
    if key_type == "PRI":
        fields.append(f"{field_name} [主键]")
    elif key_type == "MUL":
        fields.append(f"{field_name} [外键]")
    else:
        fields.append(field_name)
    tables[table_name] = {"name": table_name, "fields": fields}
return tables

```

🌟 优势：无需手动维护字段列表，自动感知数据库变更！

3 手动定义表间关系（补充业务语义）

⚠️ 自动提取无法识别业务逻辑关系，需人工补充！

```

RELATIONSHIPS = [
    {
        "from_table": "t_customers",
        "to_table": "t_sales_orders",
        "description": "客户创建销售订单",
        "field_relation": "customer_id → order.customer_id",
    },
    {
        "from_table": "t_sales_orders",
        "to_table": "t_order_details",
        "description": "订单包含多个明细项",
        "field_relation": "order_id → detail.order_id",
    },
    {
        "from_table": "t_products",
        "to_table": "t_order_details",
        "description": "产品属于订单明细",
        "field_relation": "product_id → detail.product_id",
    },
    {
        "from_table": "t_user",
        "to_table": "t_user_qa_record",
    }
]

```

```

        "description": "用户产生问答记录",
        "field_relation": "id → record.user_id",
    },
]

```

👉 建议：description 用自然语言描述，方便大模型理解！

4 写入 Neo4j 图数据库

4.1.创建唯一性约束（防止重复）

```

def create_constraints(graph):
    graph.run("CREATE CONSTRAINT IF NOT EXISTS FOR (t:Table) REQUIRE t.name IS UNIQUE")
    print("✅ 节点唯一约束已创建")

```

4.2.创建表节点（包含字段信息）

```

def create_table_nodes(graph, tables):
    for table_name, info in tables.items():
        graph.run(
            """
            MERGE (t:Table {name: $name})
            SET t.label = $label, t.fields = $fields
            """,
            name=info["name"],
            label=table_name,
            fields=info["fields"]
        )
    print("✅ 表节点创建完成！共创建 %d 个表节点" % len(tables))

```

4.3.创建表关系（带语义描述）

```

def create_table_relationships(graph):
    for rel in RELATIONSHIPS:
        graph.run(
            """
            MATCH (from:Table {name: $from_table})
            MATCH (to:Table {name: $to_table})
            MERGE (from)-[r:REFERENCES {

```

```

        description: $description,
        field_relation: $field_relation
    }]->(to)
    """
    from_table=rel["from_table"],
    to_table=rel["to_table"],
    description=rel["description"],
    field_relation=rel["field_relation"]
)
print("✅ 表关系创建完成! 共创建 %d 条关系" % len(RELATIONSHIPS))

```

5 主函数：一键构建图谱！

```

def main():
    print("🚀 开始构建 Text2SQL 语义图谱...")

    mysql_conn = connect_mysql()
    neo4j_graph = connect_neo4j()

    try:
        # 步骤1: 获取表结构
        tables = get_tables_from_database(mysql_conn)
        print(f"🇩🇪 检测到 {len(tables)} 张数据表")

        # 步骤2: 清空旧数据 (谨慎操作!)
        print("🗑️ 清空 Neo4j 中的旧数据...")
        neo4j_graph.delete_all()

        # 步骤3: 创建约束
        create_constraints(neo4j_graph)

        # 步骤4: 创建表节点
        create_table_nodes(neo4j_graph, tables)

        # 步骤5: 创建表关系
        create_table_relationships(neo4j_graph)

        print("🎉 图谱构建完成! 现在可用 Cypher 查询或供大模型调用")

    except Exception as e:
        print(f"❌ 构建失败: {str(e)}")
        raise
    finally:
        mysql_conn.close()
        print("🔌 数据库连接已关闭")

```

```
if __name__ == "__main__":  
    main()
```

6 根据表名称查询关系

```
from py2neo import Graph  
import os  
  
def test_get_table_relationships():  
    """  
    从 Neo4j 图数据库中查询预定义表之间的 REFERENCES 关系。  
    表名已写死在代码中，数据库连接信息从环境变量读取。  
  
    :return: list of dict, 每个 dict 包含 from_table, relationship, to_table  
    """  
    # 从环境变量读取数据库配置  
    NE04J_URI = os.getenv("NE04J_URI", "bolt://localhost:7687")  
    NE04J_USER = os.getenv("NE04J_USER", "neo4j")  
    NE04J_PASSWORD = os.getenv("NE04J_PASSWORD", "neo4j123")  
  
    table_names = ["t_customers", "t_sales_orders", "t_products", "t_order_details"]  
  
    # 连接图数据库  
    graph = Graph(NE04J_URI, auth=(NE04J_USER, NE04J_PASSWORD))  
  
    # Cypher 查询语句  
    query = """  
    MATCH (t1:Table)-[r:REFERENCES]-(t2:Table)  
    WHERE t1.name IN $table_names  
      AND t2.name IN $table_names  
      AND t1.name < t2.name  
    RETURN  
      t1.name AS from_table,  
      r.field_relation AS relationship,  
      t2.name AS to_table  
    """  
  
    # 如果没有表名，直接返回空结果（虽然这里不会为空）  
    if not table_names:  
        return []  
  
    # 执行查询  
    try:  
        result = graph.run(query, table_names=table_names).data()
```



```
print("查询结果: ", result)
return result
except Exception as e:
    print(f"[ERROR] 查询图数据库失败: {e}")
    return []
```

PASSED [100%]查询结果: [{'from_table': 't_customers', 'relationship': 'customer_id referer

完整代码：关注我的开源实战项目 **sanic-web**

已经集成neo4j/text2sql

如果你正在寻找一个开箱即用、支持全链路开发的大模型应用脚手架，欢迎关注我的开源项目：

GitHub 项目 → <https://github.com/apconw/sanic-web>

项目亮点

-  集成 MCP 多智能体架构
-  支持 Dify / LangChain / LlamaIndex / Ollama / vLLM / Neo4j
-  前端采用 Vue3 + TypeScript + Vite5，现代化交互体验
-  内置 ECharts / AntV 图表问答 + CSV 表格问答
-  支持对接主流 RAG 系统与 Text2SQL 引擎
-  轻量级 Sanic 后端，适合快速部署与二次开发

运行效果:

数据问答

数据问答

🗨️ 结语：让 Text2SQL 不再“瞎猜”

用 Neo4j 构建语义图谱，本质是为大模型提供结构化知识库。不再依赖模糊的 Schema 描述，而是通过“图关系”让模型理解“谁关联谁”、“怎么关联”。

📌 关注公众号【大模型应用开发实战】

👉 阅读下方置顶评论拉你进【技术支持群】



大模型应用开发

喜欢作者

[Text2SQL · 目录](#)

[下一篇 · 用 BM25 + 中文分词实现 Text2SQL 表过滤 —— 让大模型只看“相关表”，准确率...](#)

修改于2025年10月8日